Week-6

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Answer: The role of the following in a Spark application is given below:

1. Driver: The Driver is the brain of the Spark application. It runs main program, create tasks, schedule their execution and create instructions for cluster nodes.
2. Cluster Manager: The Cluster Manager allocates computing resources to the Spark applications across the cluster. Examples include YARN, Mesos, Kubernetes etc.
3. Executor: Executors are worker processes which performs tasks assigned by driver. It actually perform computation and stores data in memory or disk.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Answer: Spark's Lazy Evaluation means that transformations are not executed immediately. Instead, Spark creates a logical execution plan (Directed Acyclic Graph) and waits until action is triggered. This improves performance when chain-processing large datasets because Spark can optimize the entire sequence of transformations, eliminate unnecessary operations, and reduce data movement. As a result, only the required computations are executed, leading to better resource utilization and faster execution.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Answer: df=spark.read.csv("data/source.csv", header=True, inferSchema=True)

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer: CSV is a row based file format where data is stored row by row in plain text, whereas Parquet is a columnar file format where data is stored column by column in a compressed binary format.

The difference affects the performance because Spark often reads only a few required columns from large datasets. With Parquet, Spark can only reads only the needed columns , reducing I/O , memory usage and processing time. In contrast, csv requires reading the entire row even when only a few columns are needed. Therefore, Parquet generally proves faster query performance and better storage efficiency than CSV.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Answer: df=df.select(“product_id”, “price”)\

.filter(df.category==’Electronics’)

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Answer: df=df.withColumnRenamed(“old_name”, “new_name”) \

.withColumn(“price”, df.price.cast(“double”))

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Answer: Spark uses lineage graph that records all the transformations applied to RDD or DatFrame. If a worker node fails and some dat pertition is lost , Spark uses the lineage information to recomputed only the lost partitions from original daa and previous transformations, rather than recomputing the entire dataset. This provides fault tolerance and allows application to continue processing without data loss.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Answer: df_orders=df_orders.filter(

(df_orders.status==’Completed’) &

(df_orders.amount>1000)

)

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Answer: Predicate Pushdown is an optimization technique in Parquet where filter conditions are pushed down to storage layer. Instaed of loading all rows into memory and then applying filters, Parquet uses metadata to identify and read only the data blocks that satisfy the filter condition. This reduces disk I/O memory usage and processing time , improving query performance.

Q10: Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

Answer:df=df.withColumn(“final_price”, df.base_price*1.18)

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Answer:Transformations: Transformations are operations that creates a new RDD or dataFrame from an existing one without modifying the original data. They are lazy evaluated, meaning they are not executed until an action is triggered.

Ex: map(), filter()

Actions: Actions are the operations that trigger the execution of transformations and return a result or write data to storage.

Ex: count(), collect().

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Answer:
```
df=spark.read.parquet("path/to/input") \
        .filter(df.user_id.isNotNull()) \
        .write.csv("path/to/output", header=True)
```

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Answer: In Client Mode, the Driver program runs on the machine that submits the Spark application, while the Executors run on the cluster nodes. The client machine must remain available during execution.

In Cluster mode, The Driver program runs inside the cluster on a worker node along with the Executors. The application continues running even if the client disconnects after submission.

Thus, Client Mode is useful for development and debugging, whereas Clustur Mode is prefeered for production environments.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
df=df.filter((df.region==’North’) | (df.priority==’High’))
```

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer: It is safer to use .show(5) because it displays only a small number of rows from the dataset, reducing memory usage on the Driver. In contrast, .collect() retrieves the entire dataset from all Executors and loads it into the Driver's memory. On a multi-terabyte dataset, this can cause excessive memory consumption, slow performance, or even crash the application with an OutOfMemory error.